

Prediction Algorithm Accuracy Report

Backtests and Permutation Tests Against Live Season Data

Sports Analytics Project

September 2026

Abstract

This report backtests all four prediction algorithms currently running in the sports analytics platform against the project’s live database: 1,321 completed NBA games (2025–26 season, regular season and playoffs), 530 players, and 33,590 individual player-game boxscore rows. For each algorithm we report a walk-forward accuracy metric (predictions built only from information available strictly before the event being predicted, matching how each model is actually used in production), a comparison against a naive baseline, and a permutation test establishing whether the observed accuracy reflects real signal rather than chance. All four algorithms beat their naive baselines by a statistically significant margin ($p < 0.001$ in every case), but the size of that edge varies enormously — from decisive (win probability) to marginal (point margin). We close with a per-algorithm assessment of where accuracy plausibly tops out given more data and modeling effort, and what is worth prioritizing next.

Contents

1	Scope and Method	2
1.1	What is being evaluated	2
1.2	Data	2
1.3	Walk-forward evaluation, not in-sample evaluation	2
1.4	Metrics	2
1.5	Permutation tests	3
2	Results	3
2.1	Elo win probability	3
2.2	Four Factors margin	4
2.3	Recency-weighted fantasy points (optimizer)	5
2.4	Recency-weighted scorer points (game detail page)	5
3	Cross-Cutting Observations	6
4	Where Each Model Can Realistically Go	6
4.1	Elo win probability	6
4.2	Four Factors margin	6
4.3	Recency-weighted player point predictors (both)	7
5	Recommendations, in Priority Order	7

1 Scope and Method

1.1 What is being evaluated

The codebase has four distinct prediction algorithms, spread across a NestJS API and two standalone Python services:

1. **Elo rating** (`apps/predictor/elo.py`) — predicts home team win probability for a game.
2. **Four Factors regression** (`apps/predictor/four_factors.py`) — predicts the home-minus-away point margin for a game.
3. **Recency-weighted fantasy points** (`apps/optimizer/predict.py`) — predicts a player’s DraftKings-style fantasy point output for their next game, feeding the lineup optimizer.
4. **Recency-weighted scorer points** (`apps/api/src/games/game-detail.service.ts`) — predicts a player’s raw point total for a specific upcoming matchup, feeding the game detail page’s “predicted top scorers” list.

1.2 Data

All backtests ran against the project’s live Supabase-hosted Postgres database, not synthetic or seed data: 1,321 completed games spanning 2025-10-21 through 2026-06-13, 30 teams, 530 players, and 33,590 `PlayerGameStat` rows. This is a genuine full-season sample, large enough for the accuracy numbers below to be meaningful rather than an artifact of a handful of games.

1.3 Walk-forward evaluation, not in-sample evaluation

A prediction is only informative about a model’s real-world usefulness if it was built using nothing the model could not have known at the time. All four production algorithms already enforce this at the level of an individual prediction — every one of them is documented and tested (via `test_four_factors.py` and `games.e2e-spec.ts`) to never let a game’s own outcome, or a later game’s outcome, leak into that game’s prediction.

One subtlety required extra care. `four_factors.py`’s regression path (§2.2) fits a single ordinary-least-squares model once, using every game in the season, and then applies that one fitted model to predict every game — including games early in the season, whose prediction is therefore informed (through the fitted regression weights, not through any single feature value) by results that happened after them. This is leak-free per game but not a true walk-forward backtest. We report both numbers: the *as-shipped* evaluation (exactly what `predict_games.py` currently produces) and a *true walk-forward* evaluation that periodically refits the regression using only strictly-past games. The gap between them tells us how much the as-shipped number is inflated by full-season fitting.

The other three algorithms (Elo, and both recency-weighted point predictors) are walk-forward by construction — Elo’s rating update is inherently sequential, and the recency-weighted average at game i only ever consumes games $0, \dots, i - 1$ for a given player or team.

1.4 Metrics

- **Win probability (Elo):** Brier score (mean squared error between predicted probability and the 0/1 outcome; 0 = perfect, 0.25 = uninformative coin flip), log loss, and thresholded accuracy (predict the team with $P > 0.5$).

- **Margin (Four Factors):** mean absolute error (MAE) and root-mean-squared error (RMSE) in points.
- **Player point predictors:** MAE and RMSE in points (or fantasy points), each compared against an unweighted running mean of the same prior games — this isolates whether *recency weighting specifically* adds value over simply averaging a player’s history.

1.5 Permutation tests

For each algorithm we constructed a null distribution appropriate to how that specific model consumes its data, then computed a p-value as the fraction of null-model runs that matched or beat the real model’s observed accuracy:

- **Elo:** re-ran the entire sequential Elo forward pass 1,000 times on the real schedule, but with each game’s outcome replaced by an independent coin flip. This tests whether Elo’s Brier score reflects real team-strength signal, not just structural artifacts of the home-court constant or schedule.
- **Four Factors:** shuffled the realized-margin labels across games 1,000 times (keeping the leak-free feature matrix fixed) and refit the regression on each shuffled copy. This tests whether the Four Factors $\rightarrow$ margin relationship is real signal or an artifact of fitting three free parameters against noise.
- **Recency-weighted predictors:** shuffled each player’s own game order 500 times (destroying any genuine hot/cold-streak or role-change structure while preserving each player’s overall scoring level) and recomputed the recency-weighted model’s edge over the naive running mean on the shuffled sequences. This tests whether the recency-weighting edge is really about recency, or would appear even against a randomly ordered history.

2 Results

2.1 Elo win probability

Metric	Value
Games evaluated	1,321
Brier score	0.2146
Log loss	0.6179
Accuracy (threshold 0.5)	65.2%
Naive baseline (“home team always wins”)	55.5%
Mean predicted $P(\text{home win})$	0.601

Table 1: Elo win-probability backtest, as-shipped (walk-forward by construction).

A Brier score of 0.2146 against the 0.25 coin-flip baseline, and 65.2% thresholded accuracy against a 55.5% naive-home-favorite baseline, is a real and useful edge for a model that uses only game scores — no injuries, rest, travel, or player-level information at all.

Calibration. Binning predictions into deciles and comparing mean predicted probability to mean actual win rate in each bin shows good calibration in the upper range (0.6–0.9 predicted probability bins track actual outcomes closely) but a moderate overconfidence problem in the 0.2–0.5 range,

where the model’s predicted probabilities run consistently higher than the observed win rate (e.g. the 0.4–0.5 bin predicts 0.454 on average but the true rate in that bin was 0.359).

Predicted range	n	Mean predicted	Mean actual
0.1–0.2	5	0.173	0.000
0.2–0.3	27	0.257	0.037
0.3–0.4	87	0.357	0.230
0.4–0.5	181	0.454	0.359
0.5–0.6	308	0.554	0.451
0.6–0.7	384	0.644	0.646
0.7–0.8	230	0.746	0.765
0.8–0.9	92	0.839	0.848
0.9–1.0	7	0.920	0.857

Table 2: Elo calibration, 10 equal-width probability bins.

The consistent pattern across the 0.2–0.6 range (predicted probability higher than actual outcome) suggests the home-court-advantage constant (`HOME_COURT_ADVANTAGE_ELO`, set to 75 and borrowed from FiveThirtyEight’s published range rather than fit to this league/season) may be somewhat too large for the current data, or that `K_FACTOR` (set to 20) is letting ratings drift further from 1500 than the data actually supports before they’ve had enough games to justify it.

Permutation test. Across 1,000 randomized-outcome reruns of the same schedule, the null Brier score distribution had mean 0.2668 and standard deviation 0.0034; the observed Brier score of 0.2146 was better than every single one of the 1,000 null runs ($p < 0.001$). Elo’s edge is real signal, not an artifact of the home-court constant or schedule structure alone.

2.2 Four Factors margin

Metric	As-shipped (full-season fit)	True walk-forward
n	1,305	1,239
MAE (points)	12.52	12.65
RMSE (points)	15.80	15.94

Table 3: Four Factors margin backtest. Walk-forward refits the regression every 82 games using only strictly-past data; the small gap (+0.13 MAE) shows the as-shipped full-season fit is only mildly optimistic.

Model	MAE (points)
Four Factors regression (as-shipped)	12.52
Naive baseline (predict leaguwide average home margin)	13.36

Table 4: Four Factors vs. the simplest possible baseline.

This is the weakest result of the four. The fitted regression beats the naive “always predict the leaguwide average home margin” baseline by only 0.84 points of MAE — about a 6% improvement. The permutation test confirms this small edge is still statistically real (observed MAE 12.52 vs.

a null distribution with mean 13.34 and standard deviation only 0.02 — every one of 1,000 label-shuffled refits performed worse, $p < 0.001$), so the Four Factors $\rightarrow$ margin relationship is not noise. But an average absolute error of 12.5 points on an NBA game (where the typical margin is well within a few possessions of a coin flip) means this number is not yet precise enough to be useful for anything beyond directional (favored-team) guidance, which the Elo win-probability output already provides more reliably.

It is worth noting that the regression path *is* live in production against the current dataset (1,305 of 1,321 games used the fitted regression, comfortably above the `MINIMUM_GAMES_FOR_REGRESSION = 30` threshold) — this is a change from the assumption in the code’s own comments, which describe the heuristic fallback as what the project’s “seed data” would exercise. With a full season loaded, the regression is what actually runs.

2.3 Recency-weighted fantasy points (optimizer)

Metric	Recency-weighted	Naive running mean
n (player-game predictions)	30,447	
MAE (fantasy points)	7.74	8.25
RMSE (fantasy points)	10.39	10.96

Table 5: Fantasy-point prediction, recency-weighted average (`RECENCY_DECAY = 0.8`, unbounded window) vs. an unweighted running mean of the same prior games.

Recency weighting improves MAE by 0.51 fantasy points over a plain running mean of the same games — a modest but consistent edge (about 6%). The permutation test, which reshuffles each player’s own game order 500 times, shows this edge disappears (and reverses to slightly negative, mean -0.277) once the real chronological structure is destroyed, with every one of 500 shuffles performing worse than the observed edge ($p < 0.001$). This confirms the model is capturing genuine within-season trend (role changes, form, minutes changes) rather than just being a differently-weighted average that happens to fit better.

2.4 Recency-weighted scorer points (game detail page)

Metric	Recency-weighted (10-game window)	Naive running mean (all prior games)
n (player-game predictions)	30,447	
MAE (points)	4.45	4.64
RMSE (points)	6.12	6.28

Table 6: Raw scoring-point prediction, capped 10-game recency window vs. unweighted running mean.

Same pattern as §2.3, smaller in absolute terms since raw points have a smaller range than the fantasy-point composite: a 0.195-point MAE edge over the naive mean, statistically real ($p < 0.001$ against a permuted-order null with mean edge -0.199).

3 Cross-Cutting Observations

- **Every algorithm beats its naive baseline by a statistically significant margin.** None of the four models' apparent accuracy is an illusion of overfitting or chance — all four permutation tests reject the null at $p < 0.001$.
- **The size of the edge varies by an order of magnitude.** Elo's edge over picking the home favorite (65.2% vs 55.5% accuracy) is large and immediately useful. Four Factors' edge over predicting the league-average margin (6% MAE improvement) is real but small enough that the current margin number is not precise enough to be a standalone product feature yet.
- **None of the four models use any player-availability, injury, rest, or travel signal.** All four are pure box-score/outcome-history models. This is very likely the single largest source of remaining error across the board, and it is untouched by any of the modeling choices evaluated here.
- **The Four Factors regression is already running on real (non-heuristic) fitted weights,** contrary to what the code comments describe as the expected state with “seed data” — worth updating those comments so they don't mislead a future reader about which code path is actually live.

4 Where Each Model Can Realistically Go

4.1 Elo win probability

Current state: a solid, well-calibrated baseline with one identifiable weakness (overconfidence in the 0.3–0.5 predicted range). Two specific, low-effort next steps:

1. **Margin-of-victory scaling on the K -factor** — already flagged as a documented future step in `elo.py`'s own comments. Blowout wins currently move a team's rating by exactly as much as a one-point win, which is known (from the wider Elo/sports-analytics literature) to leave real information on the table. This is a small, well-understood change.
2. **Re-derive HOME_COURT_ADVANTAGE_ELO and K_FACTOR from this project's own data** rather than literature defaults, given the calibration bias observed in §2.1. With 1,321 games now available, a simple grid search over a held-out validation split is enough to do this properly — there was not enough data to justify this before, there is now.

Realistic ceiling without adding new input data: Brier score in the 0.19–0.20 range (vs. today's 0.2146), comparable to published results for pure-Elo NBA models. Getting meaningfully below that requires the injury/rest/lineup signal described below, which is a different category of change, not a tuning exercise.

4.2 Four Factors margin

This is the model furthest from being a reliable product feature today. Two factors are plausible sources of the weak signal:

1. **Two of Oliver's four factors are missing by design** (offensive rebound rate and the defensive-side factors — the code's own docstring explains why, correctly, given the current schema's inability to distinguish offensive from defensive rebounds). Adding an offen-

sive/defensive rebound split to `PlayerGameStat` ingestion would let a true four-factor model be fit instead of the current three-factor offense-only version, which is the most direct way to close some of this gap.

2. **No opponent-adjustment.** The current features are each team’s own raw Four Factors averages, with no accounting for the strength of the opponents that produced those averages — a team that has played a soft schedule looks better than it is. An opponent-adjusted (strength-of-schedule weighted) version of the same three factors is a natural next iteration and does not require new data, only a different aggregation of what already exists.

Realistic ceiling: point-margin prediction in the broader sports-analytics literature (with far richer inputs — play-by-play, lineup data, injury reports) typically sits around 10–11 points MAE for NBA games, which is itself close to the theoretical noise floor of single-game outcomes. Given that, this project should treat *win probability* (Elo) as the primary product signal and treat the margin number as a secondary, lower-confidence figure until the opponent-adjustment and rebound-split changes above are made — promising it as equally reliable today would overstate what it can do.

4.3 Recency-weighted player point predictors (both)

Both player-level models show the same profile: a real but modest edge from recency weighting alone, with plenty of room to grow by adding features neither model currently has access to:

1. **Opponent defensive strength.** Neither model adjusts for who the player is about to face — a player’s predicted points are currently identical whether they are playing the league’s best or worst defense. This is likely the single highest-value addition for both models, and unlike some of the other recommendations, doesn’t require new data ingestion — opponent identity is already available on every upcoming `Game` row.
2. **Minutes/role trend.** Both models predict scoring output directly, with no separate signal for playing time. A player returning from injury with reduced minutes, or a player who has just moved into a starting role, will be systematically mispredicted by a pure recency-weighted average of their box score until enough new games accumulate to catch up. A separate minutes-trend feature (or predicting per-minute rate and multiplying by a minutes forecast) is the standard fix.
3. **Home/away split.** Neither model currently distinguishes a player’s home vs. away scoring history, despite this being one of the more well-established effects in basketball analytics.

Realistic ceiling: the specific numbers above (MAE around 7.7 fantasy points / 4.5 raw points) are unlikely to improve dramatically from parameter tuning alone (e.g. adjusting `RECENCY_DECAY` or the game-window size) — the permutation tests show recency weighting’s contribution over a plain average is real but modest, so most of the room for improvement is in adding the features listed above, not in re-tuning the existing one-feature model.

5 Recommendations, in Priority Order

1. **Add an accuracy-tracking job.** None of these four algorithms currently have any ongoing accuracy monitoring — this report is, as far as this codebase’s test suite goes, the first time any of them have been evaluated against realized outcomes rather than just checked for data-leakage correctness. A lightweight scheduled job that recomputes Brier score / MAE against

newly completed games (reusing the scripts built for this report) would catch model drift going forward rather than requiring another one-off audit.

2. **Add opponent-adjustment to the two player-point predictors** (§2.3, §2.4) — highest expected value per unit of engineering effort, since the needed data (opponent team identity) is already in the schema.
3. **Re-tune Elo's HOME_COURT_ADVANTAGE_ELO and K_FACTOR** against this project's own 1,321-game sample instead of literature defaults, to address the calibration bias found in §2.1.
4. **Treat the Four Factors margin number as secondary to Elo's win probability** in any user-facing framing until the opponent-adjustment and offensive/defensive rebound split (§2.2) are implemented — today's margin prediction is only marginally better than guessing the league average.
5. **Update code comments in four_factors.py** that describe the regression path as unreachable with current data — it is reachable and already running.

Appendix: Reproducing These Results

All four backtest scripts import the real production modules (`elo.py`, `four_factors.py`, `predict.py`) directly rather than reimplementing their logic, so results reflect the code as actually shipped, not a paraphrase of it. Each script connects to the project's database (read-only for the backtest itself), computes the walk-forward metrics and permutation test described above, and prints a plain-text summary. Random seeds are fixed (42/43) for reproducibility of the permutation null distributions.